

COMP442 Assignment 4

Charles Philippe

March 2024

Semantic rules implemented

This image shows the list of semantic checks that we were able to achieve before the deadline for assignment 4, those that are highlighted are those that have been completed and tested, the others have not been completed (before the deadline). Those in red are the semantic checks that are incomplete or do not work.

Programming language features as implemented and tested in the assignment		
impl.	test.	
X	X	1.1 global symbol table is created
X	X	2.1 symbol table is created for classes
X	X	2.2 symbol table entry for a class is created in the global symbol table
X	X	2.3 global symbol table entry for a class contains a reference to the local symbol table for the class
X	X	3.1 class symbol table contains an entry for a class data member
X	X	3.2 function symbol table contains an entry for a function's local variable, including parameters of the function
X	X	4.1 symbol table is created for a free function
X	X	4.2 symbol table entry for a free function is created in the global symbol table
X	X	4.3 global symbol table entry for a free function contains a reference to the local symbol table for the free function
X	X	4.4 symbol table is created for a member function
X	X	4.5 class symbol table entry for a member function is created in the symbol table
X	X	4.6 class symbol table entry for a member function contains a reference to the local symbol table for the member function
X	X	6.1 [error] undeclared member function definition
X	X	6.2 [error] undefined member function declaration
X	X	8.1 [error] multiply declared class
X	X	8.2 [error] multiply declared free function
X	X	8.3 [error] multiply declared data member in class
X	X	8.4 [error] multiply declared identifier in function
X	X	8.5 [warning] shadowed inherited data member
X	X	8.6 [warning] local variable in a member function shadows a data member of its class
X	X	9.1 [warning] Overloaded free function
X	X	9.2 [warning] Overloaded member function
X	X	9.3 [warning] Overridden member function
X	X	10.1 [error] Type error in expression
X	X	10.2 [error] Type error in assignment statement
X	X	10.3 [error] Type error in return statement
X	X	11.1 [error] Undeclared variable (lookup - check for existence of local variable)
X	X	11.2 [error] Undeclared variable (lookup - if function is member function, check in the class for member used as variable)
X	X	11.2 [error] Undeclared variable (lookup - if function is member function, and its class inherits search in all super classes' tables for member used as variable)
X	X	11.2 [error] Undeclared data member (lookup - search in class table)
X	X	11.2 [error] Undeclared data member (lookup - if class inherits from other classes, search in all super classes' tables)
X	X	11.3 [error] Undeclared member function (lookup - search in class table)
X	X	11.3 [error] Undeclared member function (lookup - if class inherits from other classes, search in all super classes' tables)
X	X	11.4 [error] Undeclared/undefined free function (called from a member function or from a free function)
X	X	11.5 [error] Undeclared class
X	X	12.1 [error] Function call with wrong number of parameters
X	X	12.2 [error] Function call with wrong type of parameters
X	X	13.1 [error] Use of array with wrong number of dimensions
X	X	13.2 [error] Array index is not an integer
X	X	13.3 [error] Array parameter using wrong number of dimensions
X	X	14.1 [error] Circular class dependency (inheritance cycles)
X	X	14.1 [error] Circular class dependency (member types cycles)
X	X	15.1 [error] "." operator used on non-class type

Design

Because we are using the C programming language we do not have object oriented support, thus we do not use the visitor pattern.

We have a struct called Scope which encapsulates all of the data necessary in a single scope including the type of the scope(enum), a pointer to the parent scope, and a Symbol Table for that scope. We use a hash table for the symbol tables, this allows for very fast lookup speeds ($O(1)$) while doing the semantic checks, at the cost of allocating more memory than is needed by the table at a given time. The Symbol Table can be of 4 types of entries, variable entries, function entries, class entries and function parameter entries.

Class entries will have links to all of the inherited scopes, as well as it's own class scope. Function entries will have a link to it's own scope. Variable and function parameter entries are essentially the same except that function parameters will have no visibility.

The following structs show the design of the Symbol tables / Scope data structure that we used.

```
typedef enum ScopeType { GLOBAL_SCOPE, FUNCTION_SCOPE, CLASS_SCOPE } ScopeType;

typedef enum EntryType {
    VARIABLE_ENTRY,
    FUNCDEF_ENTRY,
    CLASS_ENTRY,
    FPARAM_ENTRY,
    EMPTY_ENTRY,
} EntryType;

typedef enum LangType {
    NONE_TYPE,
    VOID_TYPE,
    INT_TYPE,
    FLOAT_TYPE,
    ID_TYPE
} LangType;

typedef enum visibility { none, priv, pub } varvis;
typedef struct TypeInfo {

    LangType type;
    const char *typeString;
    int *arraydims;
    int numberofdims;

} TypeInfo;

typedef struct VariableEntry {
```

```

TypeInfo type;
const char *name;
varvis vis; // If there is no visibility we set this to none type.

} VariableEntry;

typedef struct FparamEntry {

    TypeInfo type;
    const char *name;

} FparamEntry;

typedef struct FuncdefEntry {

    varvis vis;
    bool defined;
    bool memberFunc;

    TypeInfo returnType;
    const char *name;
    struct TableEntry **fparamslist;
    int numfparams;
    struct Scope *scope;

} FuncdefEntry;

typedef struct ClassEntry {

    const char *name;
    struct Scope *scope;
    struct Scope **inheritedScopes; // Links to each of the inherited scopes.
    int inheritsCount;

} ClassEntry;

typedef struct Scope {

    ScopeType type;
    struct Scope *parentScope;
    const char *scopeName;
    struct TableEntry *entries; // We hold the Symbol table in a Scope , the
                                // symbol table will be a hash table.

} Scope;

```

```
typedef struct TableEntry {

    EntryType tableType;
    Scope *scope;

    union {
        VariableEntry varEntry;
        FuncdefEntry funcEntry;
        ClassEntry classEntry;
        FparamEntry fparamEntry;
    } data;

} TableEntry;
```

We construct the Symbol Table in a single pass through the AST, when we encounter certain node types we trigger semantic actions that will build the Symbol Table by gathering relevant information in the AST.

Because of the structure of the AST we fill out the free functions first then we fill out the structs and then finally the impl blocks get traversed, and we fill the member functions. We also perform some semantic checking during this phase we ensure that variables are declared before they are ever used wherever they are used.

We perform a second pass that does semantic checks, where we perform more in depth semantic checks like type analysis etc.

Finally we perform a traversal through the Scopes/Symbol tables themselves doing this we deal with semantic actions like ensuring no circular inheritance, function and variable shadowing.

We have a file called SemanticErrors.c with corresponding header that deals with the error reporting, we create funtions that perform the semantic checks and return specific error codes we use these during the second pass through the AST, these are used to produce specific error messages. We accumulate the messages in a dynamic array and then we sort the array by line number and print them, we also show the number of errors in the source file.

The structure of having multiple passes allows for more in depth type checks than can be done with a single traversal.

Tools used

For this section we did not make use of many tools, only using the graphviz DOT viewer to make sure that the trees are correct and comparing the Symbol tables generated to the trees that get generated. The main tool that was used was the debugging tool for C LLDB (Low Level Debugger, part of LLVM), this was used to find debug segmentation faults and other bugs encountered while developing the semantic checks.